FULL STACK DEVELOPMENT WITH MERN

Project Documentation

1. Introduction

Project Title: ShopEZ: A Seamless MERN-Stack E-Commerce Application

2. Project Overview

Purpose: ShopEZ is a production-ready e-commerce platform designed to offer a seamless online shopping experience while maintaining robust, synchronized management pipelines for administrative oversight.

Features:

- **Product Catalog:** Enables dynamic multi-criteria item discovery with real-time searching and filtering.
- **Interactive Cart:** Facilitates secure, persistent state checking and validation during checkout routines.
- **Admin Control:** Incorporates a secure dashboard workspace for product CRUD operations and tracking.

3. Architecture

Frontend: Implemented as a Single Page Application using React.js, utilizing Context API for data flow control and React Router for view navigation.

Backend: Built with Node.js and Express.js, adopting a Controller-Service pattern to handle RESTful endpoint pipelines.

Database: Relies on MongoDB via Mongoose object modeling to strictly enforce user, product, cart, and order document schemas.

4. Setup Instructions

Prerequisites: Node.js (v16+), npm, and a running MongoDB instance.

Installation:

1. Clone repository and install server files:

```
git clone https://github.com/engineering-sandbox/shopez-mern.git
cd shopez-mern/server && npm install
```

2. Install client files:

```
cd ../client && npm install
```

3. Setup configurations. Create a `.env` file in the `server` directory:

```
PORT=5000
MONGO_URI=mongodb://localhost:27017/shopez
JWT_SECRET=secure_key
```

5. Folder Structure

Client: Nested inside `/client/src` :

```
├── components/     # Reusable UI elements
├── context/        # State handlers
├── pages/          # Application views
└── App.js          # Core routes
```

Server: Contained within the `/server` root:

```
├── config/         # Database hooks
├── controllers/    # Request handlers
├── models/         # Mongoose schemas
└── routes/         # Express endpoints
```

6. Running the Application

Frontend: Run inside the client root to mount the view on port 3000:

```
cd client && npm start
```

Backend: Run inside the server root to initialize the api listener on port 5000:

```
cd server && npm start
```

7. API Documentation

Method	Endpoint	Description	Sample Response
`POST`	/api/auth/login	Authenticates credentials.	`{ "token": "ey...", "user": { "role": "customer" } }`
`GET`	/api/products	Fetches catalog records.	`[ { "_id": "p1", "name": "Wireless Mouse", "price": 29.99 } ]`
`POST`	/api/orders	Generates active transaction details.	`{ "orderId": "ord9912", "status": "Pending" }`

8. Authentication

Security uses stateless JSON Web Tokens (JWT) passed via the `Authorization: Bearer` header block. The server verifies tokens using custom gateway middleware, routing valid contexts into role-based permissions paths to guard administrative catalogs.

9. User Interface

Built with responsive layouts optimized across device form factors. The core application interface covers three specific modular workspaces: the public shopping showcase grid, the interactive checkout wizard, and the administrative dashboard table.

10. Testing

- **Backend:** API endpoint behaviors are validated via manual Postman route sequencing.
- **Database:** Validations ensure correct indexing constraints and prevent schema corruption.
- **Frontend:** Manual acceptance passes verify application layout flows and route guards.

11. Known Issues

- **Race Conditions:** Rapid near-simultaneous product acquisitions can occasionally cause brief low-stock miscounts.
- **State Reset:** Hard manual browser reloads cause a temporary layout flicker while the client session memory is verified.

12. Future Enhancements

- **Webhooks:** Introduce automated transactional mailing updates.
- **Gateways:** Switch out client payment mock objects for active Stripe API gateways.